Top-K Document Recommender with Cosine Similarity

Activity 3 · Student: Andres [add last name and student ID] · Software Development Engineering

1. Description of the approach

I implemented **Path A (TF-IDF)** in Python with scikit-learn. The repository is a simulated corpus of 30 documents from a fictional IT services company (*corpus.csv* with id, title, text, and category): 10 Policy/Process, 10 Support/Incidents, 5 FAQ, and 5 Minutes documents, each between 57 and 78 words.

The flow is: (1) corpus loading and validation (file, columns, unique IDs, non-empty texts); (2) minimal cleaning: lowercase, no accents or symbols, English stopwords removed, and light stemming; (3) vectorization with *TfidfVectorizer* over title + text; (4) cosine similarity between the seed (or the query) and the whole corpus; (5) Top-5 sorted by score, excluding the seed document when an ID is used. The interface accepts an ID (`--id DOC001`), a free-text query (`--query`), or an interactive menu, and handles errors such as unknown IDs, empty queries, or a missing file.

2. Observed results

Test	Input	Best result (score)	Coherence
1 · Policy/Process	DOC001 (passwords and MFA policy)	DOC021 FAQ: reset my password (0.4134)	4 of 5 coherent
2 · Support/Incidents	DOC012 (VPN will not connect)	DOC022 FAQ: connect to the VPN (0.5002)	4 of 5 coherent
3 · Free text	"My account is locked after I changed my password"	DOC011 ticket: user locked out (0.4455)	Top 3 relevant, last 2 noise

In all three tests the first result was the most useful document for the case. The weak recommendations were matches on generic words, such as DOC002 in test 1 and DOC011 in test 2. The category evaluation (`--evaluate`) showed that on average only 0.33 of the Top-5 shares the seed's category (Policy/Process 0.38, Support/Incidents 0.30, FAQ 0.24, Minutes 0.40). This is expected: the system groups by **topic** (passwords, VPN, backups) rather than by **document type**, which is useful because it brings together the policy, the FAQ, and the tickets about the same subject.

An extra query, "I can't log in after changing my password", exposed the main limitation: the word *log* pulled in documents about log files (DOC020, DOC030) and *change* pulled in change management (DOC004, DOC028). Only the first result was correct.

3. Areas for improvement

- **Synonyms and ambiguity:** TF-IDF compares exact words, so "sign in" and "log in", or "password" and "credentials", are different terms, and ambiguous words create false positives (extra query above). An embeddings model (Path B, for example sentence-transformers) would capture meaning better.
- **Short documents and generic words:** the false positives in tests 1 and 2 come from words common across the whole corpus. Possible fixes are domain-specific stopwords (ticket, user, system), a minimum similarity threshold, and a larger corpus.
- **Corpus bias:** the corpus is small and simulated, with many documents about passwords and VPN. Topics with few documents (printers, equipment requests) have few relevant alternatives.
- **Metadata and role-based filters:** add fields such as department, date, version, and validity, and allow filtering by category or prioritizing the latest version of a policy.
- **Security:** apply role-based permissions before recommending (do not show confidential documents to users without access) and log the queries.

- **Continuous evaluation:** collect user feedback (useful / not useful) and measure metrics such as precision@5 with a set of queries with known answers, instead of manual review.

4. Organizational reflection video

Video link (maximum 5 minutes): **[PASTE THE VIDEO LINK HERE]**

Repository: **[PASTE THE GITHUB LINK HERE]**